Sack limit W = 70

weight w = { 10, 20, 30, 40, 50 }

profit v = { 50, 75, 80, 100 }

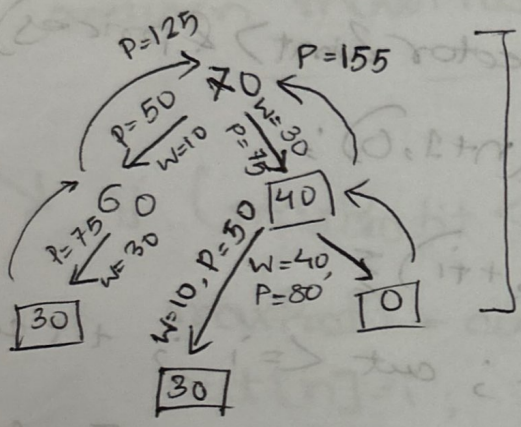
v/w 5, 2.5, 2, 2

W = 10 + 30

V = 50 + 75 = 125

But 30, 40 would give the best answer. So greedy algo. does not work.

In greedy, choice done so that best optimum solution found. But for this problem, have to traverse every choice to get the best solution Like the rod cutting.



this way all possible ways checked

n = no of items

~~n starts from 0.~~

i = = n

To keep track

```
knapsack01 (capacity, i, W, v) {
    if (i == n) return 0;
    if (dp[i][capacity] != -1) return dp[i][capacity];
    int a = KnapSack01 (capacity, i+1, w, v);
        if (capacity ≥ w[i])
            int b = KnapSack01 (capacity - w[i], i+1,
                                w, v) + v[i];
    dp[i][capacity] = max(a,b);
    return max(a,b);
```

Example:

capacity = 70
w = 40
remaining 30

=> KnapSack (30, 1, w, v)

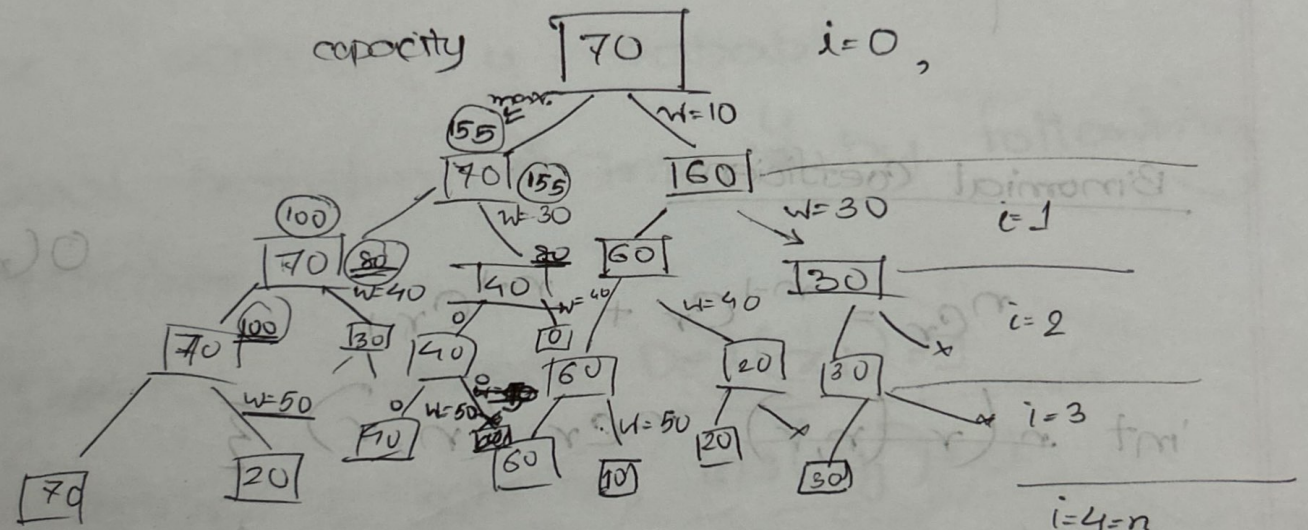
a = " (30, 2, w, v)

b = " (0, 3, w, v)

d[1] [30] = max(a, b)

Recurrence Tree :

top-down approach



Iterative Approach

W = 7

w = {1, 3, 4, 5}

v = {50, 75, 80, 100}

dp[c-W[i]][i-1] + v[i]

capacity →

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	50	50	50	50	50	50	50
2	0	50	50	75	125	125	125	125
3	0	50	50	75	80	130	80+50	75+80
4	0	50	50	75	80	100	150	75+80

teams ↓

→ check this row.

Binomial Coefficient

16/7/25

$O(n \times r)$

$$nC_r = {}^{n-1}C_r + {}^{n-1}C_{r-1}$$

```
int ~~nCr(n,r)~~ nCr(n, r) {
    if n == 0 || r == 0 return 1;
    if (dp[n][r] != -1) return dp[n][r]
    return dp[n][r] = nCr(n-1, r) + nCr(n-1, r-1);
```

Iterative version:

```
for(n=0; n<=N; n++){
```

```
for (r=0; r≤R; r++){
    if (n==0 || r==0) dp[n][r] = 1;
    else
        dp[n][r] = dp[n-1][r] + dp[n-1][r-1];
}
return dp[n][r]
```

Minimum Edit distance

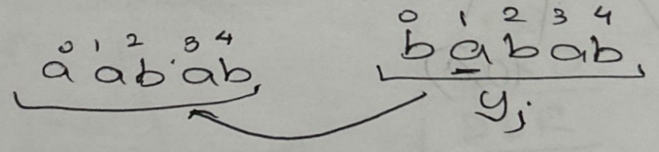
x = "aabab", y = "babab"

Goal transform x into y by following operation

1. Deletion from x, $Del(x_i) = 1$ cost
2. Insertion y_j into x, $Ins(y_j) = 1$ cost
3. change/replace x_i with y_j, $C(x_i, y_j) = 2$ cost

Minimize the total cost?

example:



1. del (x_2) = aaab, 2. ins $(y_1, 1)$ => a b a a b
3. change (x_3, y_4) => a a b b b

of used.

```
int Editdistance(i,j)                                   {
if(i==-1) return j+1 ;//Ins   if (xi==yj)                 if dp[i][j]!=-1
if (j==-1) return i+1; //Del   Editdistance(i-1,j-1);      return dp[i][j]
```

a) → Editdistance(i-1,j) + Del(xi)

b) → Editdistance(i,j-1) + Ins(yj)

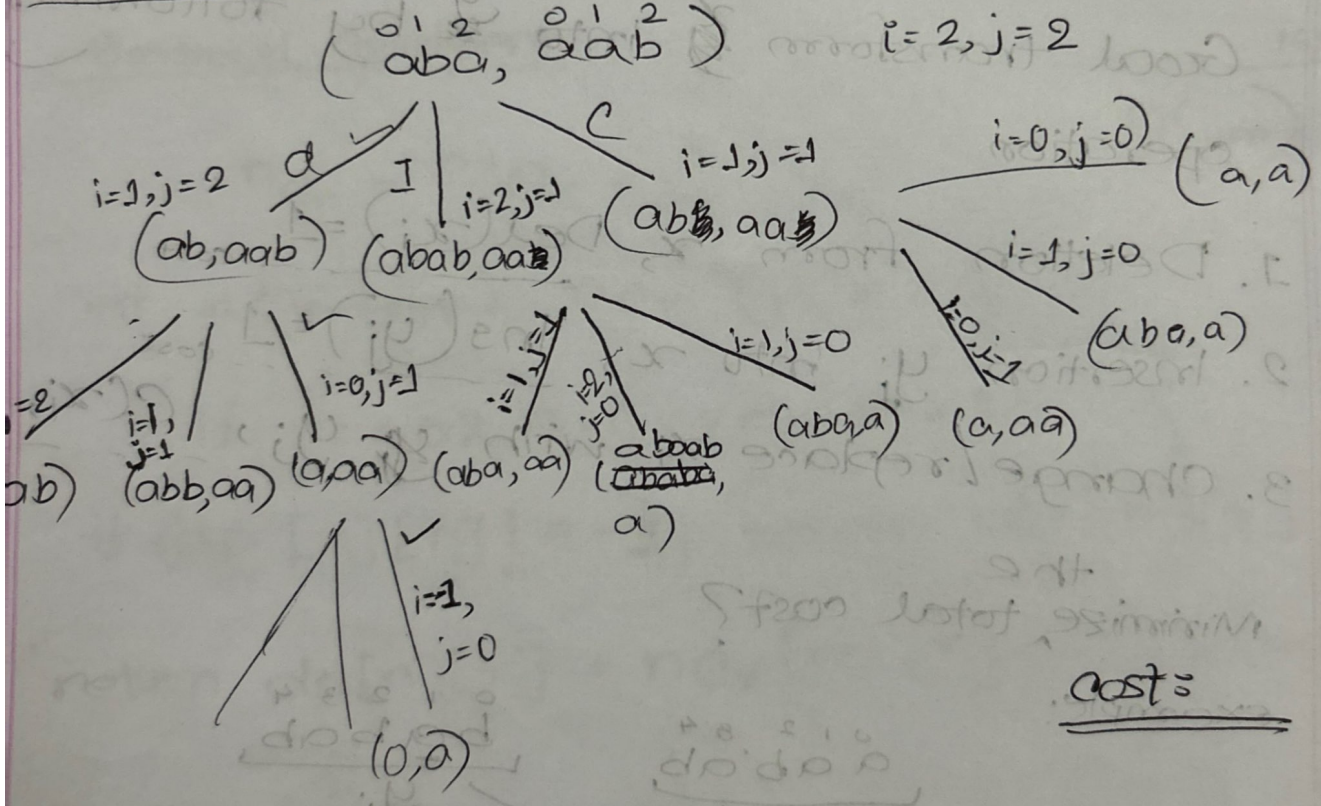
c → Editdistance(i-1,j-1) + C(xi, yj); //change/replace

dp[i][j] = [ ]

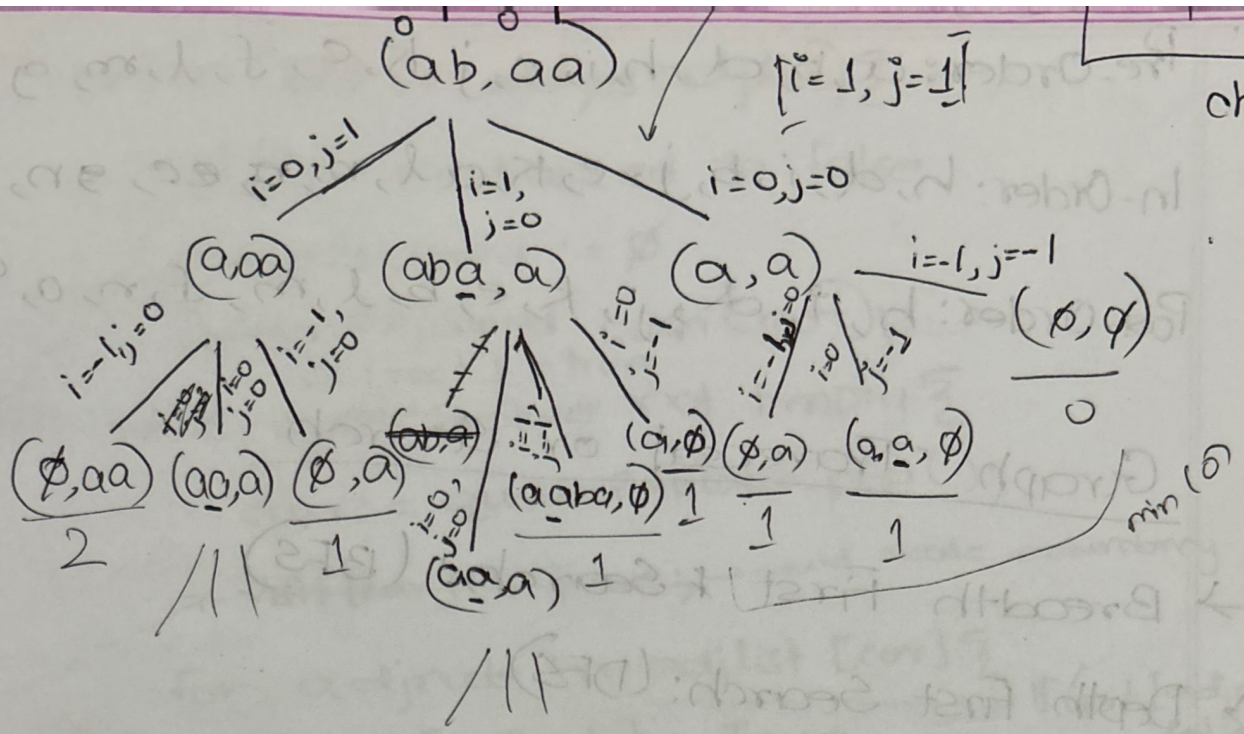
return (min(a,b,c);

return dp[i][j] = min({a,b,c});

Stimulation:



cost=



23/7/25

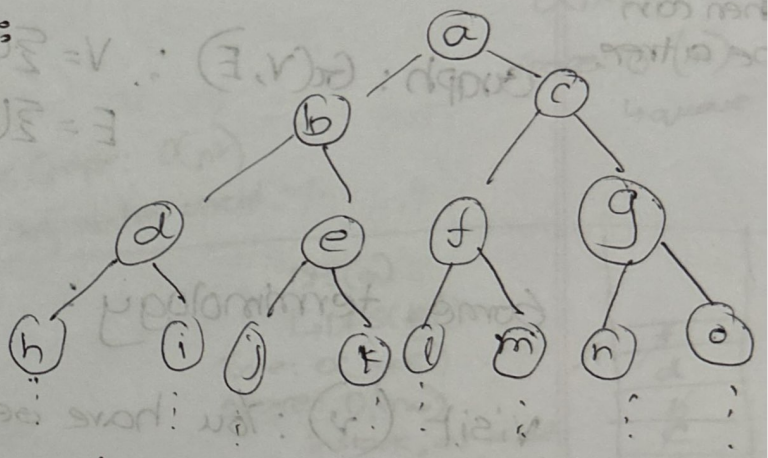
Basic Traversal and Search

Binary Tree Traversal:

(i) Pre Order [D, L, R]

(ii) Inorder [L, D, R]

(iii) Post Order [L, R, D]



Sequence of Procession

L, D, R

- go to left
- process the current node → print(cur);
- go to right

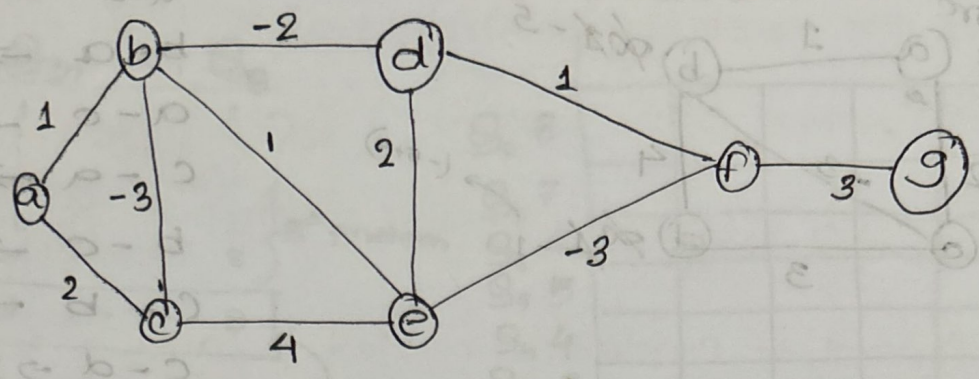
LDR
LRD
DLR
DRL
RDL
RLD

Normally L → R considered in BST.

Bellman-Ford (SSSP)

shortest Path becomes $-\infty$.

29/7/25



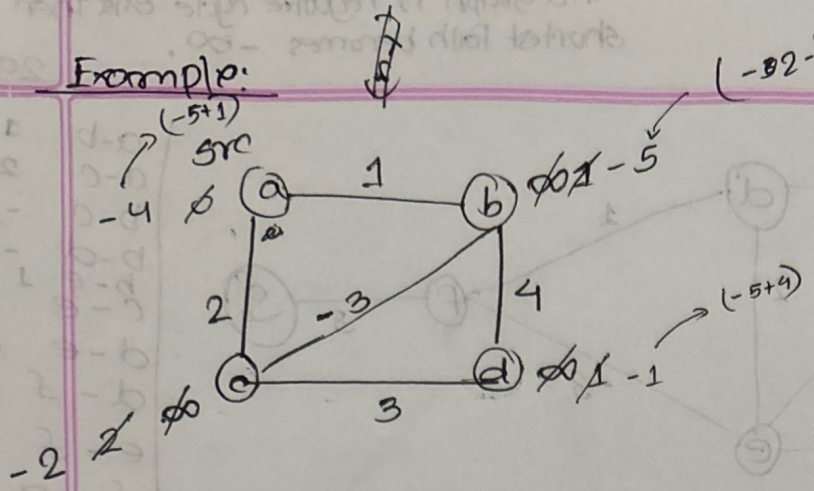
a-b	1
a-c	2
b-c	-3
b-d	-2
b-e	1
c-e	4
d-e	2
d-f	1
e-f	-3
f-g	3

```
Algorithm Bellmanford (src, edgelist){
    dist[1...n] = ∞
    dist[src] = 0;
    for (i=1 to n-1){
        for (edge : edglist){
            if (dist[edge.u] < ∞){
                dist[edge.v] = min(dist[edge.v],
                    dist[edge.u] + edge.w);
            }
        }
    }
    for (edge : edglist){
        if (dist[edge.u] < ∞){
            if (dist[edge.v] < dist[edge.u] + edge.w){
                print (Negative cycle detected with
                    edge u-v);
            }
        }
    }
```

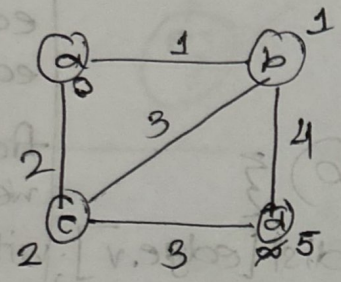
edge.u
edge.v
edge.w

from u to v, we have a path with weight w.

Example:



a-b → 1
b-a → 1
a-c → 2
c-a → 2
b-c → -3
c-b → -3
c-d → 3
d-c → 3
b-d → 4
d-b → 4



BackTracking

→ General Technique not algorithm:

→ finds a solution with n-tuples $\{x_1, x_2, x_3, \ldots, x_n\}$

x_i = part of a set S_i.

a objective function $P(x_1, x_2, x_3, \ldots, x_n)$ → maximize / minimize / find a solution (criterion function)

satisfy

And some constraint on individual x_i

and or relations among $x_i, x_j, x_k, \ldots$

The 8 queen problem:

2 Queens cannot have same column

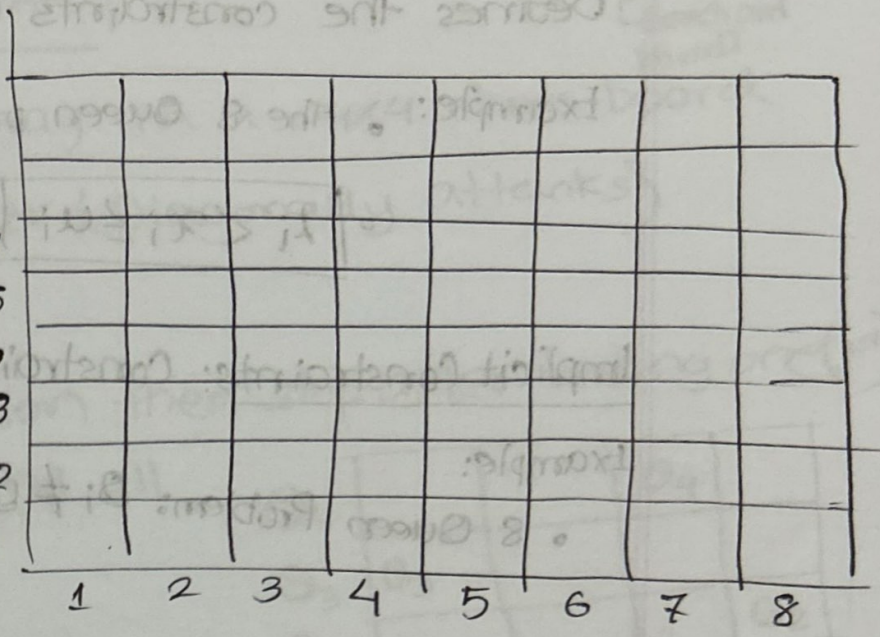
$Q_1, Q_2, Q_3, \ldots, Q_8$

possible columns: 1 1 1 ... 1 / 2 2 2 ... 2 / ... / 8 8 8 ... 8 — 8^8 problem

column assigned to each Queen: 1 2 3 8

ans. permutation of this

now 8! problems, to find a solution.



$Q_i : \{1 \ldots 8\}$

Subset Sums $W = \{w_1, w_2, w_3, \ldots w_n\}$, value V

finding K tuples, $(x_1 \ldots x_k)$

$x_i \in W$ indexes

$\{j \mid j$ is a index in $W\}$

3-tuple $x_1 = 1, x_3 = 3, x_3 = 5$

constraint → $w_1 + w_2 + w_3 = V$

Explicit Constraints 30/7/25

Defines the constraints on individual x_i based on the problem.

Example: • The 8 Queen problem: $Q_i \in \{1, 2, \ldots, 8\}$

• $l_i < x_i \leq u_i$;

Implicit Constraints: Constraints related to / among x_i.

Example:

• 8 Queen Problem: $Q_i \neq Q_j$ where $i \neq j$

$x_i \in S_i$, m_i is the size S_i

$(x_1, x_2, \ldots, x_n) \rightarrow m = m_1 m_2 m_3 \cdots m_n$

$m_1, m_2 \quad m_n$

for 8 Queen: $8, 8, \ldots, 8 \rightarrow 8^8$

$x_1 = \{1, 2, 3\}$
$x_2 = \{a, b\}$
$x_3 = \{x, y, z\}$

[1, a, x]
[1, a, y]
1, a, z
1, b, x
⋮

Solution Technique: The tuples/vectors are build by one by one variables and uses a modified $P_i(x_1, x_2, \ldots, x_i)$ to test if the subpart $(x_1, x_2, \ldots, x_i)$ leads to a valid solution.

$P_i(x_1, x_2, \ldots, x_i)$ I have found it doesn't lead to a valid solution then I will backtrack. I will not assign values to $x_{i+1} \cdots x_n$.

This will save $(m_{i+1} m_{i+2} \cdots m_n)$ number of searching tuples.

Live node: all children not expanded (visited) yet.

Dead node: all children of a node expanded

Kill node: when a node violates a constraint, its children not expanded

anymore and we kill that node.

Tree Organization

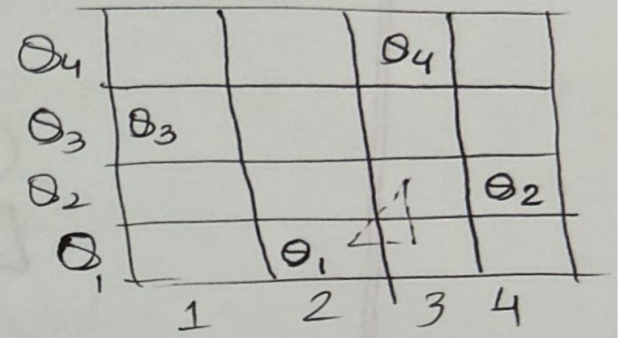
Recursive Using BFS / DFS ✓

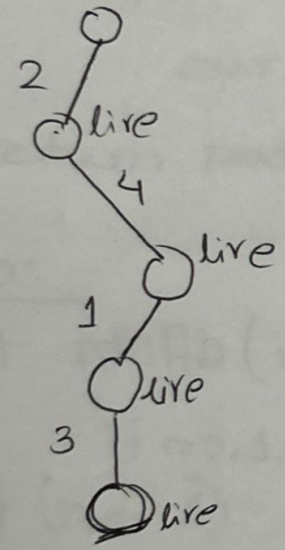
[Branch and bound]

- **4 Queen Problem:** | # Pics of tree |

Places 4 Queens in such a way in a 4x4 chees board so they do not attack each other (Diagonally attacks)

If ~~teh~~ task says to find 1 solution then stop after finding one (1st valid solution. Else continue to find all valid solution.





$$|\theta_i - \theta_j| \neq |i - j|$$

constraint for diagonal